HACK THE CONSTRAINT | THE OTHER SIDE

MedBridge

Constraint & Adaptation Report

Team X-OR | SRMIST | Harsh Chaudhary, Shadman Md, Harsh Jain

github.com/harv1036/MedBridge

173	1 of 20	51	0
automated tests, all passing	simultaneous claims wins; 19 get a 409	deliberate code breakages the tests caught	LLM calls, anywhere

1 The problem we picked

A patient leaves a clinic with one medicine list and a hospital with another, and the two rarely agree. A dose differs. A medicine is simply absent. Two different brand names turn out to be one drug. Someone has to reconcile them, and the dangerous part is the guess: assuming the newer sheet is right, or assuming a medicine that vanished was stopped.

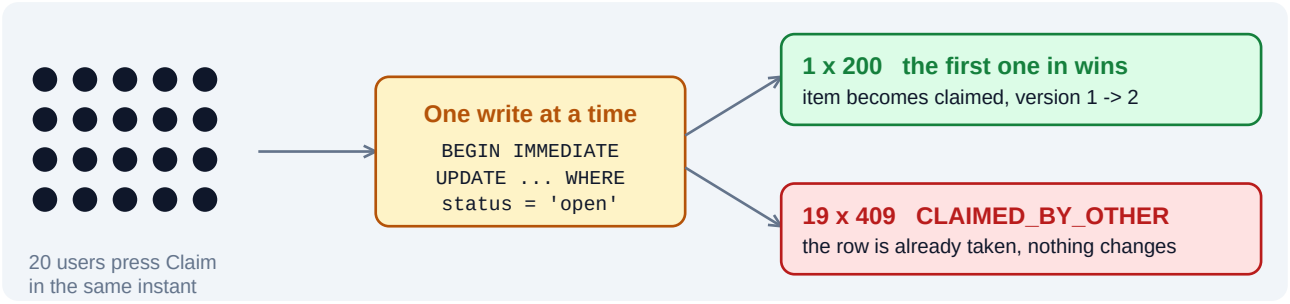
MedBridge is our answer to PS4, The Forgotten Prescription. It reads each document, turns every medicine line into a record, maps brands to generics (Glycomet and Metformin are one drug) and compares the records with plain rules. Differences land in a queue. A pharmacist claims one, reads both original lines side by side, and records a decision. The system never writes a prescription, never swaps a medicine and never suggests a dose. It points at differences and keeps the evidence. We seeded four patients, each built to show one behaviour, and an extraction step that is deliberately plain code, because an LLM that fills a blank is exactly the failure this problem is about.

2 The two constraints, as we understood them

Constraint 9. Several people act on the same item at the same moment, and requests get retried or sent twice. Two pharmacists pressing Claim together must not both succeed. A double-click must not record a decision twice.

Constraint 12. The system knows which actions are possible, but one fact it needs in order to choose is missing. The easy way out is to pick the likeliest action. We decided the system should instead stop, say what it lacks, and ask.

3 Constraint 9: making simultaneous clicks safe



Twenty users claim the same item at once. Requests pass through one write lock; the first changes the row and every later one finds it already taken.

The claim is one statement, `UPDATE ... WHERE status='open'`, run inside `BEGIN IMMEDIATE`. Whoever the database serves first changes one row; everyone after changes none and is told who holds the item. We never read the status and then write it, because that gap is exactly where two people slip through.

Each item carries a version that rises with every change, and a decision has to quote the version the reviewer was looking at. A reviewer working from a stale screen gets `STALE_VERSION` and nothing is written.

Retries use an `Idempotency-Key`. The first request stores its result in the same transaction as the change itself, so a repeat with the same key gets that stored answer back, marked `Idempotent-Replay`, and nothing runs twice. If the browser sends no key we derive one from who, what, which item and the body, so a plain double-click is covered as well. Reusing a key for a different request is refused. Uploading the same document again is stopped by a unique hash, and re-running the comparison cannot add rows because each discrepancy has a unique fingerprint.

Every attempt, accepted or rejected, is appended to an audit log inside the same transaction. Each row's hash covers the row before it, so editing any row breaks the chain from that point on, and the verify button names the first bad row.

4 Constraint 12: refusing to guess

Each kind of difference has a short list of facts someone must supply before a record can be accepted. For a dose, frequency or route change that is the date of both records and the stated reason for the change. For a medicine that is missing it is why, and whether the patient still takes it. A blank dose, unit, frequency or date, or an extraction confidence under 0.80, blocks as well. Supporting a new kind of difference is one line in a dictionary.

When something is missing the gate answers ABSTAIN. It names what is missing, writes a question from a fixed template, lists the candidate actions with every one left unselected, and allows only three: request a fact, escalate, keep pending. Trying to accept anyway returns `422 BLOCKED_MISSING_FACT`. For Ravi's metformin the generated question is:

> *"Needed for metformin: effective date of the discharge record and reason for the change. Ask the prescriber whether 1000 mg replaced 500 mg."*

Ravi's metformin, from upload to resolution. The number is the item's version at that step.

A reviewer adds each answer as a fact with its source, and the gate runs again. Only when nothing is missing do the accept buttons unlock. The patient-facing view obeys the same rule: an unresolved medicine shows only "Needs verification with your doctor or pharmacist", with no dose. While an item is pending, every record is

listed, so the newest one is never presented as the answer.

5 What went wrong on the way

The lock failure

Our twenty-user test passed on its first run, which is the dangerous kind of pass. A heavier test, many writers plus a database reset in the middle, produced *database is locked* errors. SQLite waits for the write lock by polling, and under steady load an unlucky request can lose every poll. We fixed it in the database layer: writers inside one server process now queue on a lock before asking SQLite for `BEGIN IMMEDIATE`. The same load afterwards, about 12,700 requests, produced no errors. Because that in-process lock could hide a missing transaction, we also run T1 and T2 against two server processes sharing one database.

An argument with our own brief

One test said patient 1 becomes READY after a single fact. The gate rule also demands a change reason, and the discharge note is blank. We kept the rule. It takes two facts, and the demo shows both.

A stale list on screen

Switching patients quickly in the browser could show the previous patient's medicines, because a slow reply landed after a newer one. We only found it by clicking through the interface. The page now ignores replies from superseded requests.

Smaller fixes

The first Audit tab displayed placeholder rows and now reads the real log. Writes from the page now carry a fresh Idempotency-Key per click, and buttons stay disabled while a write is in flight. User-entered text is escaped before it reaches the page. During final checks an older server from another terminal was sharing our database file, which produced a claim we had not made, so the README now says to run one server per database.

6 Evidence

The tests start a real uvicorn process on a temporary database and release their threads together with a barrier, so requests truly overlap: the run reports 20 of 20 in flight when the first response comes back.

T1	20 simultaneous claims: 1 x 200 and 19 x 409. The audit log holds 1 claim and 19 claim_rejected. Repeated across two server processes.
T2, T3	Same resolve sent 10 times with one key, and 5 times with no key: one resolutions row each. Nine of the ten T2 responses are marked as replays.
T4	Same document uploaded 5 times in parallel: one document row, discrepancy count unchanged.
T5	Resolve with an old version: 409 STALE_VERSION, nothing changed.
T6, T7	Patient 1 abstains with date and reason missing and accept returns 422. After both facts are added with sources, the gate is READY and accept succeeds.
T8	Audit chain verifies. After one row is edited directly in the database, verify names that row.
T9	Patient 1's patient view is Unresolved and the response body contains no dose.

Beyond the nine: claim storms of 20, 50, 100 and 200 users, five rounds each, always gave exactly one winner. Eighty mixed requests at once (claims, identical resolves, uploads, reads) always left one winner, one resolution and a valid audit chain. The concurrency tests ran 30 times in a row without a failure. We broke the code 51 different ways in throwaway copies, for example removing the status guard, the version check or the gate, and the tests failed every time. We also walked the whole flow in a browser, including both Concurrency Lab tests.

7 What it does not do

There is no real login: the caller names itself in a header, and the demo reset endpoint is open, which is fine for a demo and wrong for production. Storage is one SQLite file, so this is a single-machine design. The patient documents are synthetic. The drug table has 696 rows across 12 generics and was rebuilt from the public Indian Medicine Dataset (MIT licence), because the organisers' file was not available to us, so its rows may differ from theirs. MedBridge supports a reviewer; it gives no medical advice.

8 Run it

```
pip install -r requirements.txt
python seed.py
uvicorn app.main:app
```

Then open http://127.0.0.1:8000. The README has a step-by-step demo and describes the four seeded patients. Code: github.com/harv1036/MedBridge.